



Trabalho Prático

Computação Gráfica - Fase IV

2025/2026

Grupo 50

Francisco Contente - A106846 Gustavo Braga - A107379 Lucas Pinto - A107288

Índice Geral

1. Introdução	1
2. Generator	2
2.1. Alterações no ficheiro .3d	2
2.2. Cálculo das Normais e Texturas	2
2.2.1. Plano	2
2.2.2. Box	3
2.2.3. Cone	3
2.2.4. Esfera	4
2.2.5. Cilindro	4
2.2.6. Torus	5
2.2.7. Bezier	5
3. Engine	7
3.1. Alterações nas Estruturas de Dados	7
3.2. Alterações no Parser	8
3.2.1. Parsing de Luzes	8
3.2.2. Parsing de Materiais	8
3.2.3. Parsing de Texturas	8
3.3. Iluminação	8
3.4. Materiais	9
3.5. Texturas	9
3.6. Reestruturação dos VBOs	10
4. Sistema Solar	11
5. Resultado dos Testes	13
6. Conclusão	15

Índice de Figuras

Figura 1 - Sistema Solar com Iluminação e Texturas	12
Figura 2 - Teste 1	13
Figura 3 - Teste 2	13
Figura 4 - Teste 3	14
Figura 5 - Teste 4	14
Figura 6 - Teste 5	14
Figura 7 - Teste 6	14

1. Introdução

O presente relatório foi elaborado no âmbito da unidade curricular de Computação Gráfica (CG), do 3.º ano, 2.º semestre, do curso de Licenciatura em Engenharia Informática (LEI), no ano letivo 2025/2026, e descreve o trabalho desenvolvido na Fase 4 do projeto.

Esta fase teve como principal objetivo expandir o motor gráfico desenvolvido nas fases anteriores, introduzindo suporte para iluminação, materiais e texturas, de modo a aumentar significativamente o realismo visual das cenas renderizadas. Para alcançar este propósito, tornou-se necessário calcular as normais das primitivas geométricas e realizar o mapeamento dos vértices para as respetivas coordenadas de textura.

2. Generator

2.1. Alterações no ficheiro .3d

Na fase anterior, o ficheiro de saída .3d armazenava apenas as posições dos vértices dos triângulos que compõem a geometria de cada primitiva. Para suportar iluminação e mapeamento de texturas, o formato foi estendido para incluir, por cada vértice, três componentes adicionais: o vetor normal e as coordenadas de textura.

O novo formato de cada linha de vértice passa a ser:

```
x y z nx ny nz u v
```

onde (x, y, z) representa a posição do vértice no espaço 3D, (nx, ny, nz) é o vetor normal unitário nesse ponto e (u, v) são as coordenadas de textura no intervalo [0, 1].

Esta alteração foi implementada na estrutura Point, definida em *structs.h*, que agora possui os seguintes campos:

```
struct Point {  
    float x, y, z;    // posição  
    float nx, ny, nz; // normal  
    float u, v;       // coordenadas de textura  
};
```

A função *writeVerticesToFile*, em *writeVertices.hpp*, foi atualizada para escrever todos os oito campos por vértice.

2.2. Cálculo das Normais e Texturas

Para cada primitiva foi necessário derivar analiticamente os vetores normais e definir um esquema de mapeamento UV coerente com a geometria. Nas secções seguintes descrevem-se as fórmulas adotadas para cada forma.

2.2.1. Plano

O plano é uma superfície plana no plano XZ. A normal é constante em toda a superfície e aponta na direção do eixo Y positivo para a face superior e negativo para a face inferior. Como o plano é renderizado dos dois lados, são gerados dois conjuntos de triângulos com normais opostas:

Face superior: $N = (0, +1, 0)$

Face inferior: $N = (0, -1, 0)$

O mapeamento UV é definido linearmente sobre as divisões da grelha. Para a célula na linha i e coluna j de um plano com div divisões, as coordenadas de textura dos vértices são calculadas como:

$$\begin{aligned}u &= j / divisions & (\text{direção X}) \\v &= i / divisions & (\text{direção Z})\end{aligned}$$

Desta forma, a textura cobre uniformemente toda a superfície do plano, com $(u,v)=(0,0)$ no canto $(-size/2, -size/2)$ e $(u,v)=(1,1)$ no canto $(+size/2, +size/2)$.

2.2.2. Box

A caixa é composta por seis faces, cada uma com normal constante perpendicular a essa face e apontando para o exterior. Por iteração sobre as divisões em i e j , cada célula quadrangular é subdividida em dois triângulos. As normais por face são:

$$\begin{aligned}\textit{Superior} (Y = + half): N &= (0, + 1, 0) \\ \textit{Inferior} (Y = - half): N &= (0, - 1, 0) \\ \textit{Frontal} (Z = + half): N &= (0, 0, + 1) \\ \textit{Posterior} (Z = - half): N &= (0, 0, - 1) \\ \textit{Direita} (X = + half): N &= (+ 1, 0, 0) \\ \textit{Esquerda} (X = - half): N &= (- 1, 0, 0)\end{aligned}$$

O mapeamento UV em cada face é feito de forma independente, mapeando os índices de divisão para $[0,1]$:

$$\begin{aligned}u &= j / divisions \\v &= i / divisions\end{aligned}$$

Como cada face usa o mesmo esquema, a textura é aplicada de forma uniforme em todas as faces da caixa.

2.2.3. Cone

O cone é gerado com stacks e slices. O cálculo das normais na superfície lateral requer considerar o ângulo de inclinação α entre a geratriz e o eixo Y. Sendo R o raio da base e H a altura:

$$\begin{aligned}\cos_\alpha &= H / \text{sqrt}(H^2 + R^2) \\ \sin_\alpha &= R / \text{sqrt}(H^2 + R^2)\end{aligned}$$

A normal em cada ponto da lateral, parametrizado pelo ângulo θ na fatia, é então:

$$N = (\sin(\theta) \cdot \cos_\alpha, \sin_\alpha, \cos(\theta) \cdot \cos_\alpha)$$

Este vetor é unitário por construção e aponta perpendicularmente à superfície do cone. Para o stack do ápice ($i = stacks - 1$), os pontos pA e pB coincidem em (0, H, 0), pelo que apenas se emite um triângulo por slice.

A base do cone tem normal constante (0, -1, 0) com mapeamento UV em coordenadas polares centradas em (0.5, 0.5):

$$\begin{aligned}u &= 0.5 + 0.5 \cdot \sin(\theta) \\v &= 0.5 + 0.5 \cdot \cos(\theta)\end{aligned}$$

Para a lateral, o mapeamento é:

$$\begin{aligned}u &= j / slices & (\text{ângulo } \theta) \\v &= i / stacks & (\text{altura})\end{aligned}$$

2.2.4. Esfera

A esfera é parametrizada por dois ângulos: φ e θ . O ponto na superfície é dado por:

$$\begin{aligned}x &= R \cdot \cos(\varphi) \cdot \sin(\theta) \\y &= R \cdot \sin(\varphi) \\z &= R \cdot \cos(\varphi) \cdot \cos(\theta)\end{aligned}$$

Como a esfera é centrada na origem e tem raio unitário antes de escalar por R, o vetor normal em cada ponto é simplesmente o ponto normalizado, isto é, o próprio vetor de direção sem o fator R:

$$N = (\cos(\varphi) \cdot \sin(\theta), \sin(\varphi), \cos(\varphi) \cdot \cos(\theta))$$

Este vetor é por definição unitário. O mapeamento UV mapeia θ e φ linearmente sobre as slices e stacks:

$$\begin{aligned}u &= j / slices & (\theta - \text{longitude}) \\v &= i / stacks & (\varphi - \text{latitude})\end{aligned}$$

2.2.5. Cilindro

O cilindro possui três regiões: base inferior, base superior e lateral. A normal da lateral é puramente radial no plano XZ, independente da coordenada Y:

$$N_{lateral} = (\sin(\theta), 0, \cos(\theta))$$

onde θ é o ângulo da fatia. As bases têm normais constantes apontando para fora:

$$N_{superior} = (0, +1, 0)$$

$$N_{inferior} = (0, -1, 0)$$

O mapeamento UV da lateral vai de $u = 0$ a $u = 1$ ao longo das slices e de $v = 0$ (base) a $v = 1$ (topo):

$$u = i / slices, \quad v = 0.0 \text{ (base)} / v = 1.0 \text{ (topo)}$$

Para as tampas, usa-se o mesmo esquema circular do cone, centrando a textura em (0.5, 0.5):

$$u = 0.5 + 0.5 \cdot \sin(\theta)$$

$$v = 0.5 + 0.5 \cdot \cos(\theta)$$

2.2.6. Torus

O torus é parametrizado por dois ângulos: θ (ângulo em torno do eixo central, no plano XZ) e ϕ (ângulo ao longo da secção circular do tubo). Com R o raio maior e r o raio menor, um ponto na superfície é:

$$x = (R + r \cdot \cos(\phi)) \cdot \cos(\theta)$$

$$y = r \cdot \sin(\phi)$$

$$z = (R + r \cdot \cos(\phi)) \cdot \sin(\theta)$$

A normal em cada ponto aponta radialmente para fora do centro do tubo, i.e., na direção do ponto para o centro do anel da secção:

$$N = (\cos(\phi) \cdot \cos(\theta), \sin(\phi), \cos(\phi) \cdot \sin(\theta))$$

Este vetor é unitário por construção. O mapeamento UV usa os índices de slice e stack:

$$v = i / slices \quad (\theta - \text{volta principal})$$

$$u = j / stacks \quad (\phi - \text{volta do tubo})$$

2.2.7. Bezier

Para encontrar a normal num ponto (u,v) , precisamos de calcular as derivadas parciais em ordem a u e a v .

$$\frac{\partial P(u,v)}{\partial u} = \sum_{i=0}^3 \sum_{j=0}^3 B'_i(u) B_j(v) P(i,j)$$

$$\frac{\partial P(u,v)}{\partial v} = \sum_{i=0}^3 \sum_{j=0}^3 B_i(u) B'_j(v) P(i,j)$$

A normal é então calculada pelo produto vetorial das duas tangentes, e normalizada:

$$n = \frac{\partial P(u,v)}{\partial v} \times \frac{\partial P(u,v)}{\partial u}$$

O mapeamento de texturas é feito utilizando o mapeamento direto. Como um Patch de Bezier é definido por dois parâmetros, u e v , que variam ambos no intervalo $[0,1]$, estes correspondem naturalmente às coordenadas de textura u e v

3. Engine

3.1. Alterações nas Estruturas de Dados

Material

Nova estrutura que agrega as propriedades de reflexão de superfície de um modelo tridimensional:

```
struct Material {  
    float ambient[4]   = {0.2f, 0.2f, 0.2f, 1.0f};  
    float diffuse[4]   = {0.784f, 0.784f, 0.784f, 1.0f};  
    float specular[4]  = {1.0f, 1.0f, 1.0f, 1.0f};  
    float emissive[4]  = {0.0f, 0.0f, 0.0f, 1.0f};  
    float shininess    = 32.0f;  
};
```

Light

Estrutura que define uma fonte de luz na cena:

```
struct Light {  
    std::string type;  
    float position[3] = {0,0,0};  
    float direction[3] = {0,0,-1};  
    float cutoff = 45.0f;  
};
```

Model

A estrutura representativa dos modelos 3D foi estendida para incorporar o mapeamento de texturas e o respetivo material:

```
struct Model {  
    std::string filePath;  
    GLuint vertices[1] = {0};  
    unsigned int vertexCount = 0;  
    std::string textureFile;  
    GLuint texture = 0;  
    Material material;  
};
```

3.2. Alterações no Parser

O módulo de *parsing* foi expandido para processar as novas tags XML que descrevem a iluminação, os materiais e as texturas da cena.

3.2.1. Parsing de Luzes

Foi adicionada a função *parseLights*, que percorre todos os elementos `<light>` dentro do bloco `<lights>` do XML e preenche um vetor de `Light`. O tipo de luz é determinado pelo atributo `type`, podendo ser *point*, *directional* ou *spotlight*. Consoante o tipo, são lidos os atributos de posição, direção e ângulo de corte (*cutoff*).

3.2.2. Parsing de Materiais

A função *parseMaterial* lê o bloco `<color>` dentro de cada `<model>` e extrai as quatro componentes do modelo: *diffuse*, *ambient*, *specular* e *emissive*. Cada componente é especificada em valores inteiros RGB no intervalo `[0, 255]` e convertida internamente para `[0, 1]` através da função auxiliar *byteToFloat*. O valor de shininess é lido como atributo `value` do elemento `<shininess>`.

3.2.3. Parsing de Texturas

O caminho do ficheiro de textura é lido a partir do atributo *file* do elemento `<texture>` dentro de cada `<model>`. O valor é armazenado em *model.textureFile* e o carregamento efetivo para a GPU é feito posteriormente em *initGroupBuffers*.

3.3. Iluminação

O suporte à iluminação é implementado através das funções *setupLighting* e *updateLightPositions*. A primeira é responsável pela inicialização do sistema de iluminação do OpenGL, ativando `GL_LIGHTING`, normalização automática de normais (`GL_NORMALIZE`) e definindo a iluminação ambiente global da cena. Já *updateLightPositions* é invocada em cada frame, após a atualização da câmara com `gluLookAt`, garantindo que as posições e direções das luzes permanecem consistentes no referencial da câmara.

O sistema suporta até oito fontes luminosas, mapeadas dinamicamente para os identificadores nativos `GL_LIGHT0` a `GL_LIGHT7`, conforme especificado no ficheiro XML. São suportados três tipos distintos de iluminação:

- **Luzes Posicionais (Point Lights):** emitem luz em todas as direções a partir de um ponto no espaço tridimensional. A posição é enviada para o OpenGL utilizando a componente homogênea $w = 1.0$, identificando uma fonte pontual.

- **Luzes Direcionais (Directional Lights):** simulam uma fonte infinitamente distante, como o Sol, produzindo raios paralelos. Neste caso, o vetor de direção é submetido com $w = 0.0$, sendo interpretado pelo OpenGL como uma direção no infinito.
- **Focos de Luz (Spotlights):** representam uma fonte luminosa que projeta luz num cone orientado. São configurados através da posição da fonte, vetor de direção, ângulo de corte (*GL_SPOT_CUTOFF*) e expoente de suavização (*GL_SPOT_EXPONENT*), permitindo controlar a dispersão do feixe luminoso.

3.4. Materiais

Durante o ciclo de renderização, imediatamente antes do desenho de cada modelo, são aplicadas as propriedades do respetivo material através das funções *glMaterialfv* e *glMaterialf*, configurando as quatro componentes do modelo de iluminação: ambiente, difusa, especular e emissiva, bem como o fator de brilho (*shininess*):

```
glMaterialfv(GL_FRONT, GL_AMBIENT, model.material.ambient);
glMaterialfv(GL_FRONT, GL_DIFFUSE, model.material.diffuse);
glMaterialfv(GL_FRONT, GL_SPECULAR, model.material.specular);
glMaterialfv(GL_FRONT, GL_EMISSION, model.material.emissive);
glMaterialf (GL_FRONT, GL_SHININESS, model.material.shininess);
```

3.5. Texturas

O carregamento de texturas é realizado com a biblioteca DevIL (IL), inicializada com *ilInit()* no arranque. A função *loadTexture* executa os seguintes passos:

1. Gera e vincula um identificador de imagem DevIL com *ilGenImages* e *ilBindImage*.
2. Carrega o ficheiro com *ilLoadImage*.
3. Converte a imagem para o formato *IL_RGBA* com componentes do tipo *IL_UNSIGNED_BYTE* através de *ilConvertImage*.
4. Transfere os dados para a GPU com *glTexImage2D*, criando previamente um identificador OpenGL com *glGenTextures*.
5. Gera os mipmaps com *glGenerateMipmap*.

Após o carregamento, são definidos os parâmetros de amostragem da textura. O modo de repetição é configurado com *GL_REPEAT* nas direções horizontal e vertical, permitindo repetir a textura para coordenadas UV fora do intervalo [0,1]. Para filtragem, utiliza-se *GL_LINEAR* na ampliação e *GL_LINEAR_MIPMAP_LINEAR* na minificação.

3.6. Reestruturação dos VBOs

Cada modelo é carregado para a GPU num único VBO com formato de vértice intercalado, onde cada vértice ocupa 8 floats consecutivos em memória:

$$[x \ y \ z \ | \ nx \ ny \ nz \ | \ u \ v]$$

Os dados são enviados uma única vez na inicialização através de *initGroupBuffers*, que percorre recursivamente todos os grupos da cena e carrega os vértices lidos do ficheiro .3d para o respetivo VBO via *glBufferData*.

Durante a renderização, em *renderGroup*, são ativados os *client states* necessários (*GL_VERTEX_ARRAY*, *GL_NORMAL_ARRAY* e *GL_TEXTURE_COORD_ARRAY*) e configurados os respetivos ponteiros (*glVertexPointer*, *glNormalPointer* e *glTexCoordPointer*) com um *stride* fixo de $8 * \text{sizeof}(\text{float})$, indicando ao OpenGL como interpretar o formato intercalado dos vértices.

4. Sistema Solar

Nesta fase, o sistema solar desenvolvido anteriormente foi reutilizado e expandido com o objetivo de testar e demonstrar as novas funcionalidades adicionadas à engine, nomeadamente a iluminação e o suporte a texturas.

A estrutura geral da cena foi mantida, continuando a ser gerada automaticamente através do ficheiro `solar_generator.cpp`. No entanto, foram agora adicionadas texturas específicas para os diferentes planetas, Lua, cinturão de asteróides, cometa e anel de Saturno, permitindo obter um resultado visual mais detalhado e realista.

Relativamente à iluminação, foram utilizadas duas fontes de luz distintas:

- uma luz pontual colocada na posição do Sol;
- uma luz direcional complementar para melhorar a perceção visual das superfícies dos modelos.

As primitivas utilizadas nesta cena foram geradas através dos seguintes comandos:

```
.\generator sphere 1 20 20 sphere.3d
.\generator torus 1.2 0.2 20 20 torus.3d
.\generator box 2 10 box.3d
.\generator cone 1 2 10 10 cone.3d
.\generator comet patches/comet.patch
.\generator bezier patches/comet.patch 15 comet.3d
```

Para gerar o XML final do sistema solar e visualizá-lo, devem ser executados os seguintes comandos, na pasta **build/Release**:

```
.\generator solar solar_fase4.xml scenes/belt.xml
.\engine scenes/solar_fase4.xml
```

Obtém-se assim um sistema solar dinâmico, iluminado e texturado, demonstrando a integração das funcionalidades desenvolvidas nesta fase. Foi ainda incluído um vídeo demonstrativo do resultado final, disponível em **[videos/solar_fase4.mp4](#)**.

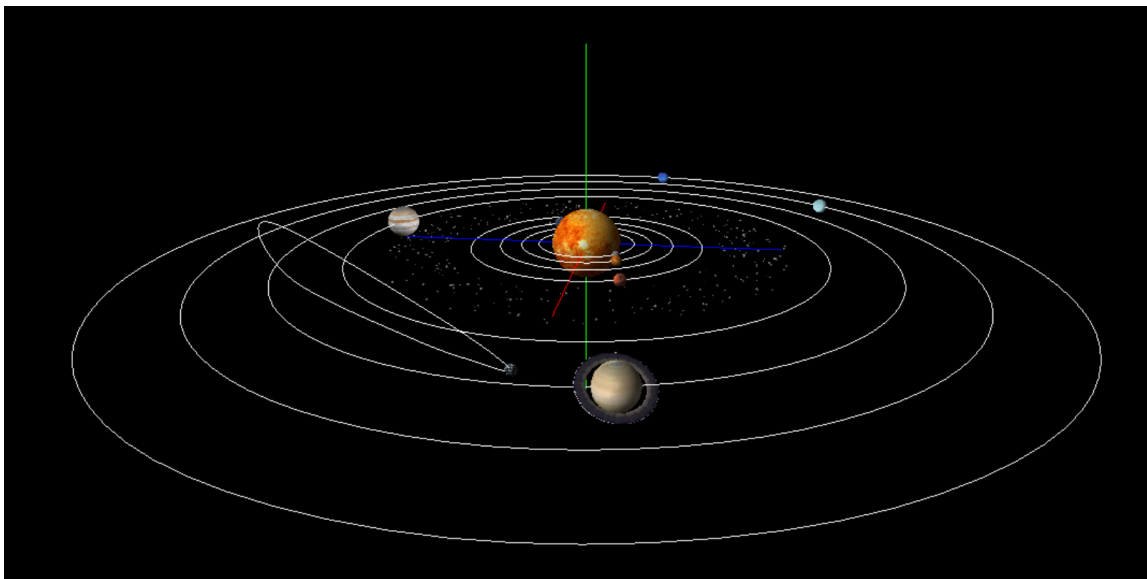


Figura 1 - Sistema Solar com Iluminação e Texturas

5. Resultado dos Testes

Nesta secção apresentam-se os resultados obtidos na execução dos testes fornecidos para a Fase 4. Estes testes permitiram validar o correto funcionamento das novas funcionalidades introduzidas, nomeadamente o suporte a diferentes tipos de iluminação, definição de cores e brilho dos materiais e aplicação de texturas nos modelos.

Ao longo dos seis testes disponibilizados, foram verificados diferentes cenários envolvendo luzes direcionais, pontuais e do tipo *spotlight*, bem como o comportamento dos componentes *diffuse*, *ambient*, *specular* e *emissive* dos materiais. Foram também testados diferentes valores de *shininess*, permitindo validar o cálculo do brilho especular nos objetos da cena.

Adicionalmente, os testes permitiram confirmar a correta aplicação de texturas em diferentes primitivas e superfícies geradas anteriormente, demonstrando que a engine interpreta corretamente os novos elementos associados às texturas nos ficheiros XML.

Os resultados obtidos demonstram assim o correto funcionamento do sistema de iluminação e texturização implementado, verificando-se uma renderização consistente das cenas fornecidas nos testes da Fase 4.

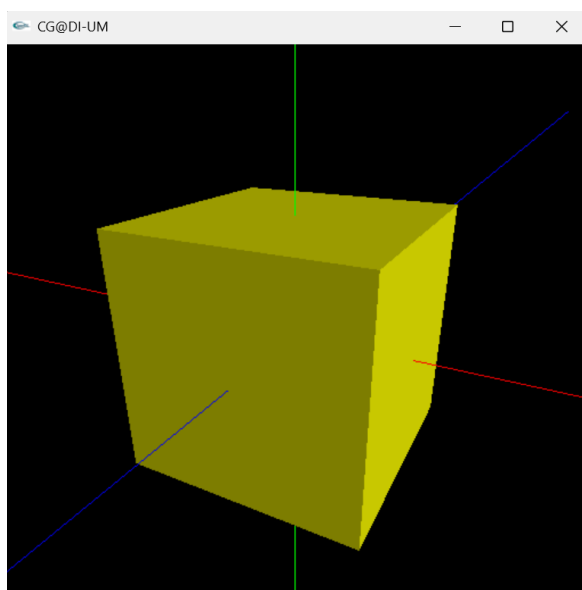


Figura 2 - Teste 1

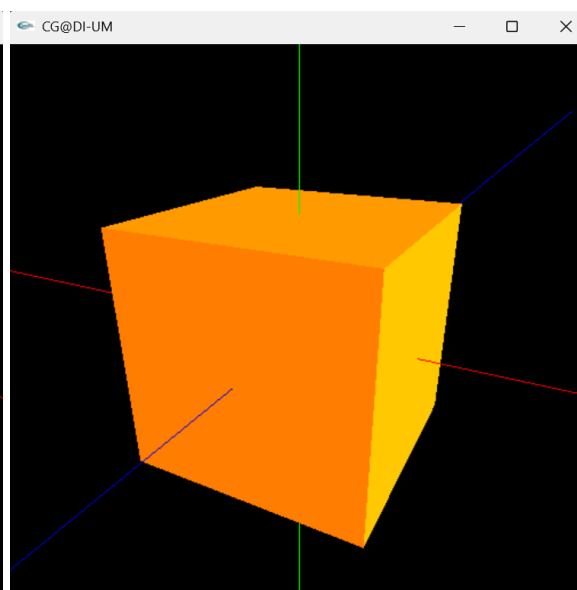


Figura 3 - Teste 2

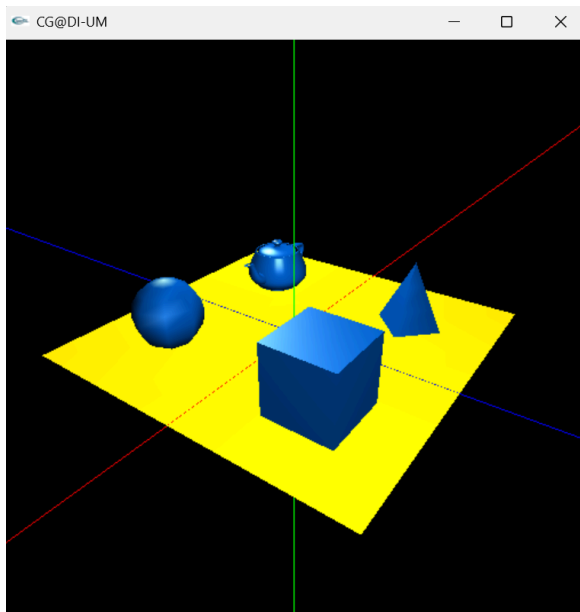


Figura 4 - Teste 3

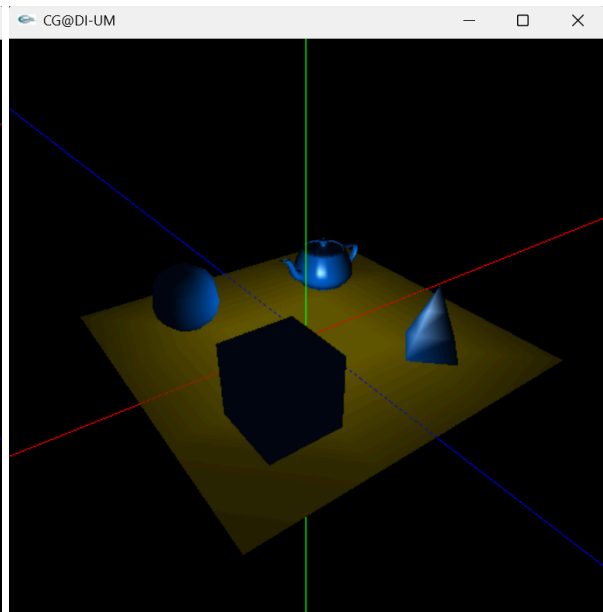


Figura 5 - Teste 4

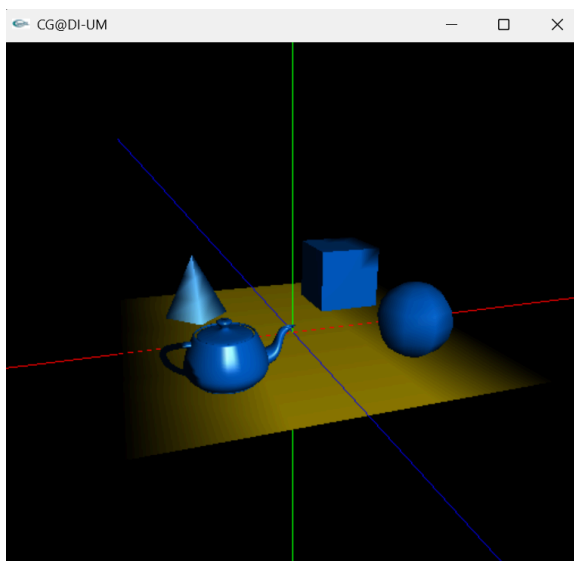


Figura 6 - Teste 5

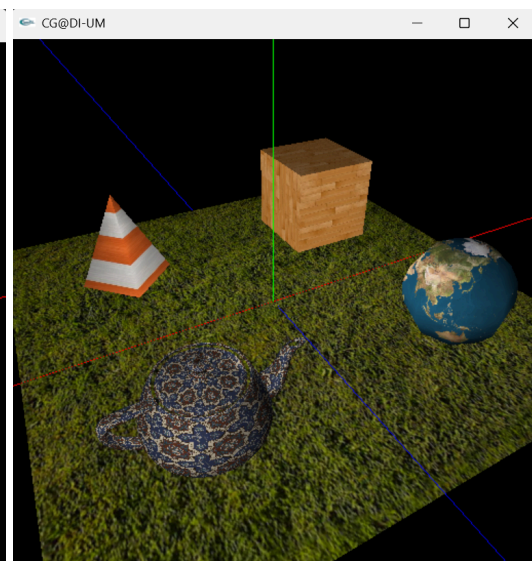


Figura 7 - Teste 6

6. Conclusão

Nesta quarta e última fase do projeto foram implementadas as funcionalidades necessárias para suportar iluminação, materiais e texturas na engine desenvolvida ao longo do semestre. Os principais objetivos definidos para esta fase, nomeadamente a geração de normais e coordenadas de textura no generator, o suporte a diferentes tipos de luzes e materiais, e a aplicação de texturas nos modelos, foram concretizados com sucesso.

A integração destas funcionalidades permitiu aumentar significativamente o realismo visual das cenas renderizadas, possibilitando a interação dos modelos com diferentes fontes de luz e a utilização de texturas para enriquecer a sua aparência. A validação foi efetuada através dos testes fornecidos para a fase, complementados por diversos testes desenvolvidos ao longo da implementação, para serem verificadas individualmente as diferentes componentes introduzidas ao longo do desenvolvimento. Foi ainda realizada a adaptação da cena do sistema solar, onde foi possível demonstrar de forma integrada todas as funcionalidades implementadas.

Considerando o projeto na sua totalidade, desde a primeira fase até à sua conclusão, o desenvolvimento deste trabalho permitiu consolidar conhecimentos fundamentais de Computação Gráfica, incluindo modelação geométrica, transformações hierárquicas, animação baseada em curvas, superfícies paramétricas, otimização da renderização com VBOs, iluminação e mapeamento de texturas. A implementação progressiva destas funcionalidades contribuiu para uma melhor compreensão da arquitetura de um motor gráfico tridimensional e dos principais conceitos envolvidos na construção de aplicações gráficas interativas.

Considera-se, assim, que todos os objetivos propostos para o projeto foram atingidos com sucesso, resultando numa engine funcional capaz de representar e animar cenas tridimensionais com um nível de realismo significativamente superior ao da versão inicial.